

The Campbell Scientific PakBus Protocol

ENGINEERING SPECIFICATIONS, ARCHITECTURE, AND IMPLEMENTATION ANALYSIS

1. Protocol Overview & Architecture

The **PakBus** protocol is a proprietary, packet-switched network protocol developed by Campbell Scientific for robust communications with data loggers (such as the CR300, CR310, and CR1000X series) and peripheral devices. Engineered to operate efficiently over low-bandwidth, high-latency links like serial channels (RS-232/RS-485), RF networks, and cellular modems, PakBus features built-in routing mechanisms, multi-node multiplexing, and deterministic client-server transaction flows.

PakBus functions simultaneously across layers analogous to the OSI reference model: providing physical framing, node addressing, network routing, and application-level commands. Within a PakBus network, every device acts as a node identified by a unique 12-bit integer physical address (ranging from 1 to 4094). As observed in typical deployment topologies:

- **Client / Master Nodes** (e.g., a data collection engine or edge gateway) usually occupy a higher node ID range, such as `SRC_NODE = 0x802` (2050 decimal).
- **Server / Logger Nodes** (e.g., remote telemetry stations) default to localized or small integer addresses, such as `DEST_NODE = 0x001` (1 decimal).

2. Framing Layer & Frame Synchronization

At the serial transmission layer, PakBus encapsulates packets using dedicated control bytes and an escaping mechanism to allow asynchronous binary streaming without collision or false framing errors.

2.1 Frame Boundaries & Escaping Rules

Every raw packet is bounded at its beginning and end by a unique synchronization byte: `SYNC_BYTE = 0xBD`. To prevent data values inside the payload from being misidentified as frame boundaries, an escaping (quoting) mechanism is strictly enforced:

- If the data byte value is `0xBD`, it is transformed into a two-byte sequence: `0xBC 0xBD`.
- If the data byte value is `0xBC` (the escape control byte itself), it is transformed into: `0xBC 0xBC`.

Prior to processing transmission bytes, the receiver must read the stream sequentially, stripping out the leading and trailing `0xBD` delimiters, and unescaping any byte following a `0xBC` control marker to rebuild the pristine message payload.

2.2 Transmission Wake-up Sequence

To ensure low-power loggers transition out of sleep states into active UART receiving states, standard implementations prepend a series of synchronization bytes before initiating a frame. A standard wake pulse consists of writing 40 consecutive `0xBD` bytes to the serial line, followed by a brief settling delay (~600ms). For subsequent operational frames, a smaller preamble of 6 consecutive `0xBD` bytes is sufficient to maintain sync.

3. The PakBus Signature & Nullifier Algorithm

Error detection within PakBus relies on a specialized, stateful 16-bit signature algorithm computed across the unescaped packet header and body bytes. Unlike standard CRC-16 implementations, the PakBus signature relies on a seed value and deterministic bit-shifting combined with addition.

3.1 Signature Computation Engine

The signature initializes with a standard seed value of `0xAAAA`. For each subsequent byte in the message, the current signature state undergoes a precise operation sequence:

$$Sig_{\{new\}} = ((Sig_{\{old\}} \ll 1) \& \sim 0x01FF) + (Sig_{\{old\}} \gg 8) + Byte$$

If the temporary shifted register exceeds its bounds, adjustments ensure it fits properly into the signature tracking format. The final evaluation produces a 16-bit validation field.

3.2 Signature Nullifier Mechanics

A unique feature of PakBus framing is that the signature is not directly appended to the end of the packet for verification by comparison. Instead, the transmitter calculates and appends two specific bytes known as the **Signature Nullifier**.

The Nullifier Objective: The two nullifier bytes are calculated such that when they are included in the cumulative signature calculation by the receiving engine, the resulting final running signature value drops exactly to `0x0000`.

This allows the receiver to stream the incoming packet through the signature engine without branching. If the final signature result is exactly zero after the last byte is processed, the frame is verified as structurally valid.

4. Packet Header Structure

Every PakBus frame contains an 8-byte structured routing header. The header fields are tightly packed using big-endian byte ordering. The individual bitfields are defined as follows:

Offset / Width	Field Name	Description
Bits 0–3 (4 bits)	LinkState	

Offset / Width	Field Name	Description
		Manages link layer handshake tracking (<code>0x9</code> = Ring, <code>0xA</code> = Ready, <code>0xB</code> = Finished).
Bits 4–15 (12 bits)	DstPhyAddr	The physical address of the next hop device on the local network segment.
Bits 16–17 (2 bits)	ExpMore	Expect More flag. Non-zero indicates that subsequent fragmented data blocks follow.
Bits 18–19 (2 bits)	Priority	Packets scheduling priority structure across congested nodes.
Bits 20–31 (12 bits)	SrcPhyAddr	The physical address of the device originating the frame on the local segment.
Bits 32–43 (12 bits)	HiProto	High-level protocol group classification (<code>0x0</code> = PakCtrl, <code>0x1</code> = BMP5).
Bits 44–55 (12 bits)	DstNode	The final target destination node ID of the packet.
Bits 56–59 (4 bits)	HopCnt	Routing hop counter incremented at each network boundary.
Bits 60–71 (12 bits)	SrcNode	The original node ID that created the data payload.

5. Core Protocol Sub-Groups & Command Structures

The application layers are split based on the `HiProto` field into two distinct protocol execution subsystems: **PakCtrl (0x0)** for link maintenance and routing initialization, and **BMP5 (0x1)** (BMS Parameter Control Protocol v5) for data lookup and transfer.

5.1 PakCtrl Protocol Commands (`HiProto` = `0x0`)

- **Hello Command (0x09) & Hello Response (0x89):** A required low-level handshake used to discover and bind connection links between two nodes. It verifies node availability and sets up the transaction engine tracking layers.
- **Bye Command (0x0D):** Signals the graceful termination of a communication session, informing the target node that it can release routing or buffer resources.

5.2 BMP5 Data Engine Protocol Commands (HiProto = 0x1)

BMP5 handles data acquisition, system status discovery, and file transfer operations. Sub-commands follow a request/response structure, where the response code mirrors the command code with the most significant bit set high (e.g., Command `0x09` responds with `0x89`).

- **GetClock (0x17) & Clock Response (0x97):** Queries or synchronizes the internal real-time clock of the logger.
- **GetProgramStats (0x18) & Response (0x98):** Returns runtime tracking parameters including the logger's operating system version, OS compilation date, active data collection table metadata counts, and current compilation error states.
- **GetTableDef (0x0B) & Response (0x8B):** Allows a client to enumerate data tables. It yields the exact alphanumeric table name, table sizes, column spacing layouts, and a unique 16-bit **DefSig** (Definition Signature) that acts as a schema version hash.
- **CollectData (0x09) & Response (0x89):** The data acquisition command. On modern loggers like the CR300/CR310, the table ID passed to the collect request must be logically bitwise OR'd with `0x8000` to signal a structured collecting action.
- **FileUpload (0x1D) & Response (0x9D):** Manages block-level binary reads and writes across the logger's storage filesystems (e.g., retrieving system configurations or structured summaries like `.TDFS`).

6. Low-Level Data Encoding & Specialized Types

Data structures inside PakBus frames reflect the low-power hardware constraints of early microcontrollers, mixed with scientific precision standards.

6.1 Campbell Scientific NSec Timestamps

Timestamps are transmitted as an 8-byte structures dividing the time metric since the custom epoch of **January 1, 1990, 00:00:00 UTC**. The payload contains:

1. A 4-byte big-endian `uint32` representing total elapsed seconds.
2. A 4-byte big-endian `uint32` representing fractional elapsed nanoseconds.

6.2 Campbell Floating Point Formats (FP2 & FP4)

To preserve bandwidth, floating-point records are dynamically packed using one of two custom formats:

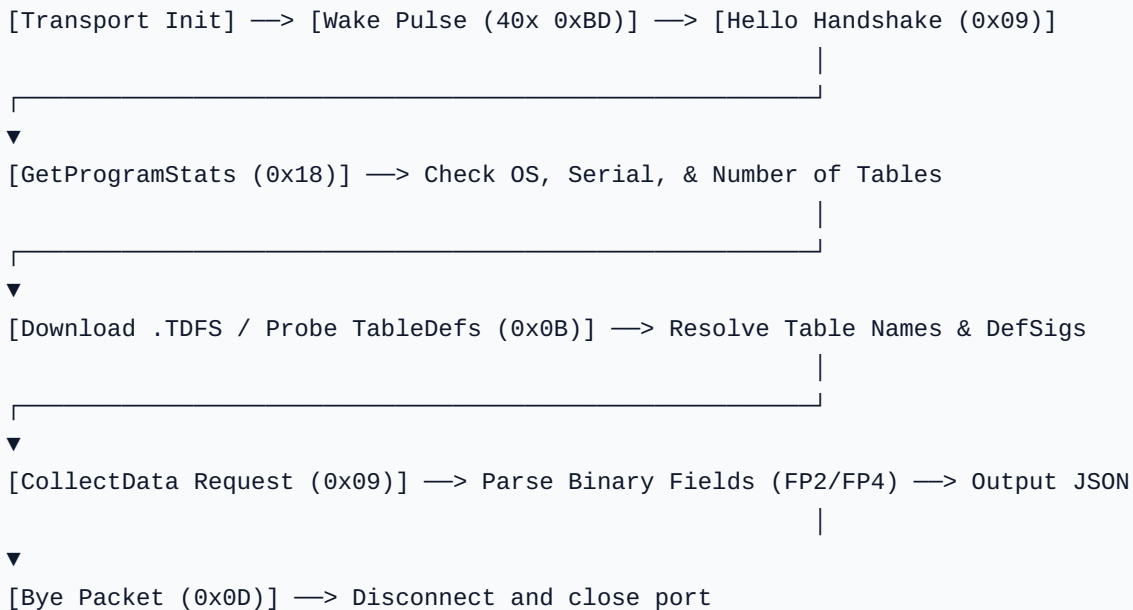
- **FP4 (Standard IEEE-754 Single Precision):** A standard 4-byte representation mapped via binary bit conversions directly into decimal floating numbers.
- **FP2 (Campbell 16-bit Float):** A compressed 2-byte representation. It packs a sign bit, a 3-bit base-10 exponent multiplier, and a 12-bit mantissa:

Value = Mantissa $\times 10^{\{(Exponent - 6)\}}$

If the sign bit is set high, the resolved numeric expression evaluates as negative.

7. Operational Sequence & Implementation Analysis

The programmatic execution of a PakBus engine follows a strict operational order:



As verified by live logger logs, an operating station (such as a CR310 tracking solar metrics at the NLCIL 50MW plant in Tamil Nadu) populates internal tables (e.g., `Min1`, `Min15`, `Hour`, `Daily`) with system status arrays. A typical payload contains real-time variables such as:

```
{
  "time": "2026-07-03T18:16:16.702000+00:00",
  "fields": {
    "OSVersion": "CR310.11.3.0",
    "OSDate": "11/19/2025",
    "SerialNumber": "7241",
    "PanelTemp": 31.3315,
    "Battery": 12.8020,
    "LithiumBattery": "OK, ON POWER UP",
    "CompileResults": "CPU:NLCIL_50MW_Tamil_Nadu_25-05.CR300 -- Compiled OK."
  }
}
```

This structured data pipeline ensures accurate environmental metrics acquisition, secure protocol execution, and reliable telemetry across complex network topologies.